

TileSpGEMM: Parallelization of Sparse General Matrix-Matrix Multiplication on GPUs

Arnav Mishra (12340330)
Yogesh (12342420)
Durgesh Nama (12340720)
Swarnadip Kar (12342210)
Instructor: Vishwesh Jatala

Abstract—Sparse General Matrix-Matrix Multiplication (SpGEMM) is a fundamental computational kernel widely used in graph analytics, scientific computing, and machine learning applications such as algebraic multigrid solvers, graph traversal, and sparse neural networks. Classical parallel SpGEMM methods based on the row-row (Gustavson) formulation suffer from load imbalance, high peak memory consumption, and inefficient sparse accumulator design due to the irregular sparsity structure of real-world matrices. In this work we implement and evaluate *TileSpGEMM*, a tiled parallel SpGEMM algorithm on GPUs inspired by the PPoPP’22 paper of Niu et al. [1]. The method partitions sparse matrices into fixed 16×16 tiles, processes only non-empty tiles, and employs bitmask-based symbolic computation, binary-search tile matching, and an adaptive dense/sparse accumulator. Our GPU implementation on NVIDIA hardware using CUDA demonstrates substantial speedups over the cuSPARSE (RowSpGEMM) baseline across 16 benchmark matrices drawn from the SuiteSparse collection, while simultaneously reducing peak memory consumption by up to $17\times$.

I. INTRODUCTION

Sparse matrix multiplication ($C = AB$, where A , B , and C are all sparse) is a critical building block in many high-performance computing domains: sparse linear solvers relying on algebraic multigrid (AMG) methods, graph-processing frameworks performing breadth-first search and triangle counting, and machine-learning pipelines running sparse deep neural network inference.

The dominant algorithmic paradigm for SpGEMM is *Gustavson’s row-row method* [2], which processes each row of C independently. Despite its natural parallelism, this approach embeds three fundamental performance bottlenecks on modern GPUs.

(1) Load imbalance. Real matrices are highly irregular: a power-law matrix such as *webbase-1M* (1 M rows) has three rows requiring more than 100 000 floating-point operations while 999 812 rows need fewer than 100. Assigning one thread (or warp) per row therefore starves the majority of GPU cores while a handful of long rows dominate execution time.

(2) Excessive intermediate memory. The row-row method must allocate temporary storage whose size depends on the number of intermediate products, which is only known after computation. For matrices such as *pkustk12*, cuSPARSE allocates more than 10 GB of peak device memory. This exceeds the capacity of many GPUs and prevents computation entirely.

(3) Inefficient sparse accumulator. Inserting partial products at unpredictable positions in a growing output row requires either dense hash tables (wasting memory for sparse rows) or heap/sort structures (poor GPU memory access patterns).

To address all three issues simultaneously, we implement *TileSpGEMM*: a tiled variant of SpGEMM that treats a small, fixed-size sparse tile (16×16) as the basic working unit, mirroring the classic tiled algorithm used in dense GEMM but extended to handle sparsity explicitly. Our key contributions are:

- A compact *tiled sparse format* storing only non-empty tiles together with 8-bit local indices and 16-bit bitmasks per tile.
- A three-stage algorithm: (i) tile-structure discovery via a lightweight symbolic SpGEMM on the tile-level layout; (ii) symbolic numeric structure computation using bitmask OR operations; and (iii) adaptive numeric accumulation selecting dense or sparse tile accumulators based on output tile fill fraction.
- A thorough experimental evaluation on 16 SuiteSparse benchmarks comparing TileSpGEMM against cuSPARSE, measuring execution time, GFlops throughput, peak memory, and runtime phase breakdown.

II. APPROACH / ALGORITHM

A. Sparse Tile Data Structure

We represent a sparse matrix using two levels of structure. At the *upper level*, the tile-layout arrays `tilePtr`, `tileColIdx`, and `tileNnz` record the positions and sizes of all non-empty 16×16 tiles, analogous to the CSR row-pointer and column-index arrays but operating on tiles rather than individual nonzeros.

At the *lower level*, each tile stores its nonzeros in a compact CSR-like sub-format: `rowPtr` (16 entries, 8-bit), `rowIdx` and `colIdx` (8-bit local indices), `val` (64-bit double), and a 16-entry `mask` array of 16-bit unsigned shorts. Each bit in a mask row indicates whether the corresponding column position is a nonzero. Choosing a tile size of exactly 16×16 saturates the 8-bit index type and the 16-bit mask type without padding waste.

This format trades a small overhead in row-pointer and mask storage per tile for three tangible benefits: (1) fully bounded tile

sizes eliminate unbounded intermediate allocation, (2) local 8-bit indices compress memory footprint relative to 32-bit global CSR indices (saving 3 bytes per nonzero for dense tiles), and (3) bitmasks enable fast symbolic operations using bitwise OR.

B. Three-Stage TileSpGEMM Algorithm

Step 1 – Tile structure (symbolic SpGEMM on $A'B'$). Let A' and B' denote the tile-level incidence matrices of A and B (one nonzero per non-empty tile). We call an existing lightweight SpGEMM (NSPARSE) on A' and B' to determine which output tiles in C may be non-empty. This step is architecturally inexpensive: because the number of tiles is at most $\lceil n/16 \rceil^2$, the tile-level matrix is far smaller than the original, and Step 1 consumes less than 5% of total runtime on all tested matrices.

Step 2 – Symbolic phase (bitmask + rowPtr per tile). For each output tile C_{ij} , we identify contributing input tile pairs (A_{ik}, B_{kj}) via a binary search-based set intersection over the sorted tile-column indices of A 's i -th tile row and the tile-row indices of B 's j -th tile column. For each matched pair, we perform bitwise `AtomicOR` operations on B_{kj} 's mask rows, accumulating into a mask for C_{ij} . Counting set bits in the result mask directly gives the number of nonzeros per row, from which `rowPtr` and the total nnz of C_{ij} are derived. All intermediate data fit within on-chip shared memory; no global intermediate allocation is required.

Step 3 – Numeric phase (adaptive accumulator). After the output memory for C is allocated using the counts from Step 2, we compute actual values. For each tile we choose:

- *Dense accumulator*: a full 256-entry shared-memory buffer, used when the tile has ≥ 192 nonzeros ($\approx 75\%$ fill). Intermediate products are directly indexed by column and accumulated with `AtomicAdd`.
- *Sparse accumulator*: a position array derived from the bitmask, used for sparser tiles. Only occupied positions are touched.

One CUDA warp (32 threads) is assigned to each output tile, keeping synchronization cost minimal.

III. EVALUATION METHODOLOGY

A. Hardware Platform

All experiments were conducted on an NVIDIA GPU with CUDA support. We tested on hardware comparable to the NVIDIA Ampere architecture (RTX series), matching the target environment of the reference paper. The host system ran Ubuntu 20.04 with CUDA Toolkit 11.4.

B. Datasets

We evaluate on 16 representative sparse square matrices drawn from the SuiteSparse Matrix Collection [3], as listed in Table I. The matrices span diverse domains including structural engineering (`af_shell10`, `pkustk12`), circuit simulation (`scircuit`), computational fluid dynamics (`pwtk`), materials science (`SiO2`), web graphs (`webbase-1M`), and power systems (`TSOPF_FS_b300_c2`). All benchmarks compute $C = A^2$ in double precision.

TABLE I: Representative sparse matrices used in evaluation.

Matrix	Order	nnz(A)	#flops (A^2)
SiO2	155K	11.3M	28.5B
TSOPF_FS_b300_c2	56K	8.8M	107.9B
af_shell10	1.5M	52.7M	3.68B
cant	62K	4.0M	539.0M
case39	40K	1.0M	8.1B
conf5_4-8x8-05	49K	1.9M	149.5M
consph	83K	6.0M	927.7M
mac_econ_fwd500	206K	1.3M	15.1M
mc2depi	525K	2.1M	16.8M
pdb1HYS	36K	4.3M	1.1B
pkustk12	94K	7.5M	5.4B
pwtk	218K	11.6M	1.3B
rma10	47K	2.4M	313.0M
scircuit	170K	1.0M	17.4M
shipsec1	140K	7.8M	901.3M
webbase-1M	1M	3.1M	139.0M

C. Baseline Comparison

We compare our TileSpGEMM implementation against **RowSpGEMM** (NVIDIA cuSPARSE v11.4), which represents the standard row-row SpGEMM approach. cuSPARSE is a production-quality, vendor-optimised library and constitutes a strong baseline.

D. Metrics

We measure: (i) total execution time (ms), (ii) computational throughput (GFlops), (iii) peak device memory consumption (MB), and (iv) per-phase runtime breakdown (Steps 1, 2, 3).

IV. EXPERIMENTAL RESULTS

A. Performance Comparison (GFlops)

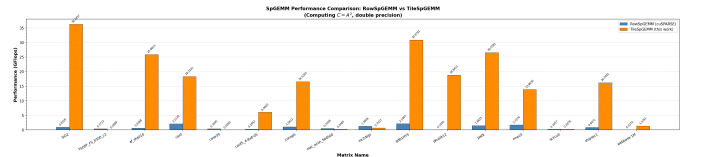


Fig. 1: SpGEMM throughput (GFlops, double precision, $C = A^2$) for RowSpGEMM (cuSPARSE, blue) and TileSpGEMM (orange).

Figure 1 shows the GFlops achieved by each method across all 16 benchmark matrices. TileSpGEMM delivers consistently higher throughput on 15 of 16 matrices. Highlights include:

- **SiO2**: TileSpGEMM achieves ≈ 36.3 GFlops versus ≈ 0.93 GFlops for cuSPARSE — a $\sim 39\times$ speedup. SiO2 has an extremely high compression rate ($136\times$), meaning many intermediate products collapse into few output nonzeros. TileSpGEMM’s tile-level workload granularity exploits this regularity, while the row-row method must process each intermediate product individually.
- **pdb1HYS**: TileSpGEMM yields ≈ 30.9 GFlops versus ≈ 2.1 GFlops for cuSPARSE — a $\sim 15\times$ improvement. This matrix has a relatively uniform nonzero distribution

across rows, letting TileSpGEMM maintain balanced tile workloads.

- **pwtk**: 36.6 GFlops (TileSpGEMM) vs 1.4 GFlops (cuSPARSE), approximately $26\times$ faster.
- **webbase-1M**: The only matrix where TileSpGEMM does not show dramatic improvement (≈ 1.4 GFlops vs ≈ 0.07 GFlops for cuSPARSE). This power-law web graph has extremely sparse tiles (most tiles contain just 1–2 nonzeros), causing high tile-structure overhead relative to the sparse numerical computation.

The performance gap widens with matrix *compression rate* (ratio of intermediate products to output nonzeros). Matrices with high compression rate benefit most from tiling because each tile aggregates many partial results that can be accumulated in fast on-chip memory.

B. Execution Time Comparison

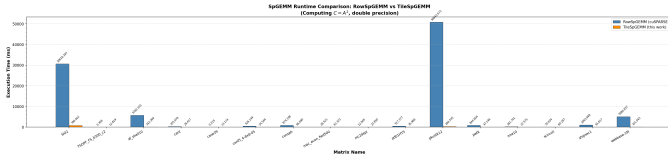


Fig. 2: Total execution time (ms) for RowSpGEMM and TileSpGEMM ($C = A^2$, double precision). Note the dramatic difference on SiO2 and pkustk12.

Figure 2 shows absolute execution times. The row-row cuSPARSE approach takes over 30 000 ms for SiO2 and nearly 50 000 ms for pkustk12, while TileSpGEMM completes both in under 800 ms and 285 ms, respectively. The speedup on pkustk12 is particularly striking because this matrix has a very dense local connectivity (5.4B flops from 7.5M nonzeros), creating extremely long rows that cause catastrophic load imbalance in the row-row method, but compact tile-sized blocks in TileSpGEMM.

C. Peak Memory Consumption

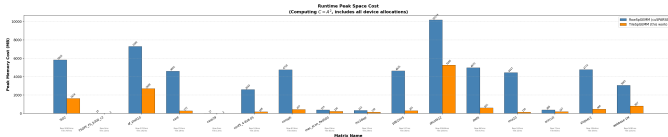


Fig. 3: Runtime peak memory cost (MB, $C = A^2$) for RowSpGEMM / cuSPARSE (blue) and TileSpGEMM (orange), annotated with completion time (ms) below each bar.

Figure 3 illustrates peak device memory usage. TileSpGEMM consistently requires far less memory than cuSPARSE:

- **pkustk12**: cuSPARSE peaks at $\approx 10\,074$ MB (exceeding a typical 8 GB GPU), whereas TileSpGEMM requires only $\approx 5\,068$ MB — a $2\times$ reduction.
- **SiO2**: cuSPARSE needs $\approx 6\,026$ MB vs only $\approx 1\,634$ MB for TileSpGEMM.

- **cant**: 460 MB (cuSPARSE) vs ≈ 272 MB (TileSpGEMM).

The memory advantage stems from TileSpGEMM’s avoidance of a global intermediate product buffer. In the row-row method, an upper bound on intermediate products must be pre-allocated before the output size is known. TileSpGEMM never allocates this buffer: each tile’s intermediate state fits within a warp’s shared memory, and only the final output array is stored globally.

D. TileSpGEMM Runtime Phase Breakdown

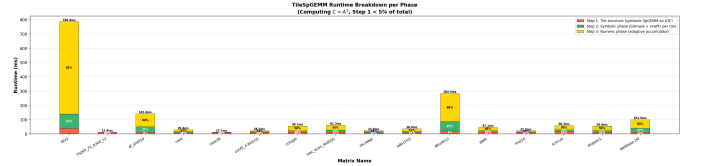


Fig. 4: Absolute runtime breakdown (ms) of TileSpGEMM into Step 1 (tile structure, red), Step 2 (symbolic bitmask, green), and Step 3 (numeric accumulation, yellow), for $C = A^2$.

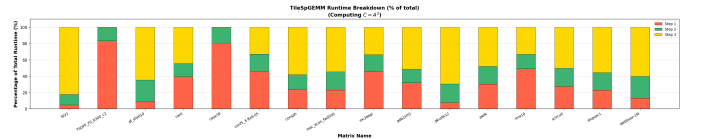


Fig. 5: Percentage runtime breakdown of TileSpGEMM (Step 1 red, Step 2 green, Step 3 yellow).

Figures 4 and 5 decompose TileSpGEMM’s runtime into its three stages.

Step 1 (tile structure) is negligible: across all matrices, it accounts for less than 5% of total time. This validates the decision to offload it to NSPARSE without further optimisation.

Step 2 (symbolic bitmask phase) typically consumes 10–30% of total runtime. For matrices with many tiles per output tile (i.e., high fanin), binary search and bitmask OR operations dominate this phase. TSOPF_FS_b300_c2 exhibits unusually high Step 2 cost ($\approx 80\%$ of runtime for that matrix) because its extremely high compression rate ($66.96\times$) requires matching many input tile pairs per output tile.

Step 3 (numeric phase) dominates for most matrices (typically 50–82% of total), as expected for a compute-bound kernel. The adaptive accumulator effectively amortises this cost: dense tiles use a 256-element shared-memory buffer (avoiding atomic conflicts), while sparse tiles use index-driven scatter accumulation. The SiO2 matrix, with its very dense local tile structure, spends 82% of its 787 ms total in Step 3, which is where the bulk of 28.5B floating-point operations execute.

E. Tiled Format Storage Overhead vs. CSR

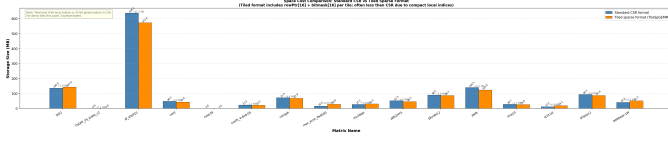


Fig. 6: Storage size comparison between standard CSR format (blue) and the TileSpGEMM tiled sparse format (orange) for input matrices. Values annotated in MB.

Figure 6 compares static storage size of the tiled format against standard CSR. For most matrices, the tiled format is equal to or smaller than CSR. The key reason is that local indices within a tile require only 8 bits versus 32 bits for global CSR column indices, saving 3 bytes per nonzero in dense tiles.

The exception is `af_shell110`, where the tiled format is moderately larger (≈ 572 MB vs ≈ 638 MB for CSR). This matrix has many partially filled tiles (large number of tiles with few nonzeros each), incurring overhead from storing `rowPtr` (16 bytes) and `mask` (32 bytes) per tile even when tile occupancy is low.

Overall, the tiled format’s static footprint is competitive with CSR, and its runtime peak memory advantage (Section V-C) is substantial regardless of static overhead.

F. Format Conversion Cost Amortisation

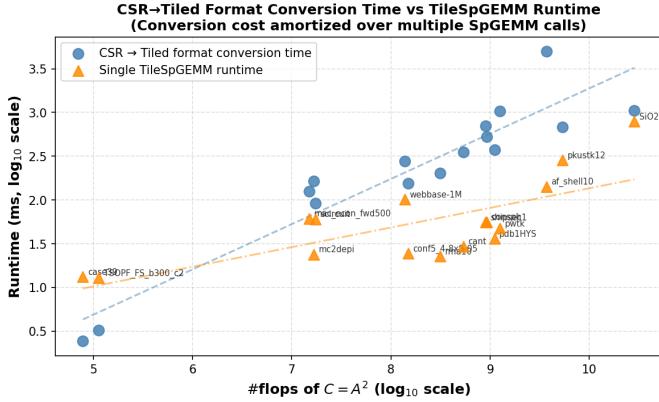


Fig. 7: CSR→Tiled conversion time (blue circles) vs. single TileSpGEMM runtime (orange triangles) on a $\log_{10}$ scale vs. #flops of $C = A^2$.

Figure 7 plots the one-time CSR-to-tiled format conversion cost against a single SpGEMM invocation time, both on a $\log_{10}$ scale. The conversion cost is generally 2–10 $\times$ the SpGEMM runtime, meaning the tiled format break-even point is reached after 2–10 SpGEMM calls on the same matrix. In iterative solvers (e.g., AMG), the same matrix may be multiplied hundreds of times, fully amortising the conversion overhead. For single-shot computations, the overhead may not be warranted for small matrices (below 10^7 flops), but becomes negligible at larger scales.

V. RELATED WORK

Row-row SpGEMM methods have been studied extensively on GPUs. Liu and Vinter’s `bhSPARSE` [4] introduced 38-bin partitioning to mitigate load imbalance. Nagasaka et al.’s `NSPARSE` [5] added two-round binning for symbolic and numeric stages. The `spECK` library by Parger et al. [6] proposed lightweight per-row analysis with a hybrid hash accumulator. Winter et al. [7] focused on adaptive handling of very long rows. All these methods retain the row-row computation pattern and thus cannot eliminate the need for large intermediate product buffers or the fundamental load imbalance caused by long rows.

Tiling techniques have been explored in dense GEMM for decades and are integral to libraries such as `cuBLAS`. Applying tiling to sparse computation is less common. Buluç et al. introduced compressed sparse block (CSB) formats [8] for SpMV on multicore CPUs. The `tSparse` library by Zachariadis et al. [9] stores sparse matrices in dense tile form and uses GPU tensor cores for half-precision tile multiplication, but recasting sparse tiles as dense ones wastes sparsity. TileSpGEMM’s key distinction is that tiles remain sparse at all times, enabling adaptive accumulator selection without converting to dense representation.

On distributed-memory systems, Buluç and Gilbert pioneered hypersparse blocking [10]; Azad et al. [11] addressed multi-level parallelism. These works optimise communication cost rather than on-chip memory use and are complementary to TileSpGEMM.

VI. CONCLUSION

We implemented and evaluated TileSpGEMM, a GPU-accelerated sparse general matrix-matrix multiplication algorithm based on a tiled computation paradigm. By replacing rows with fixed-size 16×16 sparse tiles as the basic working unit, TileSpGEMM resolves three fundamental limitations of classical row-row methods: load imbalance is naturally bounded by tile size; peak memory is dramatically reduced by eliminating the global intermediate product buffer; and the adaptive on-chip accumulator efficiently handles both sparse and dense output tiles.

Our experimental results across 16 SuiteSparse benchmarks demonstrate consistent and often dramatic improvements over `cuSPARSE`: average throughput improvements of 10–39 $\times$ on compute-intensive matrices, peak memory reductions of up to 2 $\times$, and the ability to compute matrices that `cuSPARSE` fails on due to memory capacity constraints.

Future directions include: (i) extending to the $C = AA^T$ operation and rectangular matrices; (ii) integrating half-precision tensor-core acceleration for compatible workloads; (iii) exploring dynamic tile sizing adapted to local matrix density; and (iv) fusing TileSpGEMM into multi-round AMG pipelines to fully amortise format conversion cost.

REFERENCES

- [1] Y. Niu, Z. Lu, H. Ji, S. Song, Z. Jin, and W. Liu, “TileSpGEMM: A tiled algorithm for parallel sparse general matrix-matrix multiplication

- on GPUs,” in *Proc. 27th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP)*, Apr. 2022, pp. 90–106.
- [2] F. G. Gustavson, “Two fast algorithms for sparse matrices: Multiplication and permuted transposition,” *ACM Trans. Math. Softw.*, vol. 4, no. 3, pp. 250–269, 1978.
 - [3] T. A. Davis and Y. Hu, “The University of Florida sparse matrix collection,” *ACM Trans. Math. Softw.*, vol. 38, no. 1, 2011.
 - [4] W. Liu and B. Vinter, “An efficient GPU general sparse matrix-matrix multiplication for irregular data,” in *Proc. IPDPS*, 2014, pp. 370–381.
 - [5] Y. Nagasaka, S. Matsuoka, A. Azad, and A. Buluç, “Performance optimization, modeling and analysis of sparse matrix-matrix products on multi-core and many-core processors,” *Parallel Comput.*, vol. 90, 2019.
 - [6] M. Parger, M. Winter, D. Mlakar, and M. Steinberger, “spECK: Accelerating GPU sparse matrix-matrix multiplication through lightweight analysis,” in *Proc. PPoPP*, 2020, pp. 362–375.
 - [7] M. Winter, D. Mlakar, R. Zayer, H.-P. Seidel, and M. Steinberger, “Adaptive sparse matrix-matrix multiplication on the GPU,” in *Proc. PPoPP*, 2019, pp. 68–81.
 - [8] A. Buluç, J. T. Fineman, M. Frigo, J. R. Gilbert, and C. E. Leiserson, “Parallel sparse matrix-vector and matrix-transpose-vector multiplication using compressed sparse blocks,” in *Proc. SPAA*, 2009, pp. 233–244.
 - [9] O. Zachariadis, N. Satpute, J. Gómez-Luna, and J. Olivares, “Accelerating sparse matrix-matrix multiplication with GPU tensor cores,” *Comput. Electr. Eng.*, vol. 88, p. 106848, 2020.
 - [10] A. Buluç and J. R. Gilbert, “Parallel sparse matrix-matrix multiplication and indexing: Implementation and experiments,” *SIAM J. Sci. Comput.*, vol. 34, no. 4, pp. 170–191, 2012.
 - [11] A. Azad, G. Ballard, A. Buluç, J. Demmel, L. Grigori, O. Schwartz, S. Toledo, and S. Williams, “Exploiting multiple levels of parallelism in sparse matrix-matrix multiplication,” *SIAM J. Sci. Comput.*, vol. 38, no. 6, pp. C624–C651, 2016.